# Questions

# OpenMP reduction子句的变量初始值

- 在 OpenMP 的 reduction 子句中，每个线程的私有变量副本的初始值由归约操作符的类型决定

- 初始值规则
  - 加法 (+)：初始值为 0
  - 乘法 (*)：初始值为 1
  - 逻辑与 (&&)：初始值为 1（即 true）
  - 逻辑或 (||)：初始值为 0（即 false）
  - 最大值 (max)：初始值为该数据类型的最小可能值（如 INT_MIN 对于 int 类型）
  - 最小值 (min)：初始值为该数据类型的最大可能值（如 INT_MAX 对于 int 类型）

- 假设使用 reduction(+:sum)，每个线程的私有 sum 会初始化为 0
- 在并行区域内，各线程操作自己的副本
- 退出并行区域时，所有线程的副本通过加法合并，结果再与主线程的原始值相加。

```c
#include<stdio.h>
#include<stdlib.h>
#include<omp.h>

int main(int argc,char *argv[])
{
    int sum=10; //主线程的原始值

    int thrdCnt=strtol(argv[1],NULL,10);

    #pragma omp parallel num_threads(thrdCnt) reduction(+:sum)
    {
        sum+=1;  //每个线程的私有sum初始为0，  执行后变为1
    }

    printf("Final sum = %d\n", sum);  //输出为10+4=14，  假设4个线程

    return 0;

}
```

- 执行结果：
  - 每个线程的私有 sum 初始化为 0
  - 并行结束后，各线程的 sum 相加为 4（假设 4 个线程）
  - 主线程的原始值 10 与归约结果 4 相加，最终 sum = 14

# 操作符与初始值的对应关系

- 加法（+）：
  - 私有副本初始为 0，最终结果为 原始值 + 各线程和。
- 乘法（*）：
  - 私有副本初始为 1，最终结果为 原始值 * 各线程乘积。
- 最大值 (max):
  - 私有副本初始为 INT_MIN，最终结果为 max(原始值, 各线程最大值)

Q1: OpenMP中是否也存在临界区的问题?

如何解决?

```
# pragma omp critical
  global_result += my_result ;
```

## Q2：在OpenMP中，一个变量的范围指什么？

In OpenMP, the scope of a variable refers to the set of threads that can access the variable in a parallel block.

Q4: 直接在一个循环体（loop）外面加上

```
# pragma omp parallel for
```

来实现循环体并行化是否总是可行的？会不会出错？

```
fibo[ 0 ] = fibo[ 1 ] = 1;
# pragma omp parallel for num_threads(2)
for (i = 2; i < n; i++)
    fibo[ i ] = fibo[ i – 1 ] + fibo[ i – 2 ];
```

1 1 2 3 5 8 13 21 34 55

this is correct

but sometimes we get this

1 1 2 3 5 8 0 0 0 0

- OpenMP compilers don't check for dependencies among iterations in a loop that's being parallelized with a parallel for directive.

- A loop in which the results of one or more iterations depend on other iterations cannot be correctly parallelized by OpenMP.